

It Is Not the Cryptography: A Discarded Exception Dominates Token Validation Cost in a Node.js Service

Katha Sikdar¹

¹Independent Researcher.

Contributing authors: ferdouskatha35@gmail.com;

Abstract

The per-request cost of JSON Web Token validation is routinely attributed to cryptography. We show that in a widely used Node.js library the attribution is wrong, and that the true mechanism is neither cryptography nor the conversion of key material. When the caller supplies a shared secret as a string — which is what the library’s interface invites — `jsonwebtoken` resolves it by attempting to parse it as an *asymmetric* key first, and falling back to the symmetric constructor only after that attempt throws. The conversion it falls back to costs 0.583 μ s. The discarded failure that precedes it costs 18.04 μ s, or 60% of the entire validation call. Measuring across four container runtimes on one machine, we find the magnitude of this cost is governed by the bundled OpenSSL version rather than by containerisation, virtualisation or the JavaScript engine: two runtimes sharing the OpenSSL 3.0 series but differing in V8 major version both pay roughly 373.79–420.62 μ s, while an OpenSSL 3.5 runtime pays 22.94 μ s. Everything else in the validation path varies by at most 1.26 $\times$ across the same runtimes; the discarded attempt varies by 25.36 $\times$. A survey of public code finds the avoiding pattern rare rather than absent: across four combinations of language and import form, no cell exceeds 0.0303% of files importing the library; in a sample of 327 call sites across 247 repositories, 0 pass a pre-parsed key; and a search targeted at the pattern found it genuinely used at 7 call sites across 6 projects. We propose a fix, and report a constraint on it that we believe is the more important result: the discarded attempt is load-bearing for an existing key-type defence, so the obvious form of the optimisation must not be applied.

Keywords: JSON Web Token, performance measurement, benchmarking methodology, Node.js, OpenSSL, software defect

1 Introduction

Authentication in cloud-native services is commonly described as imposing a per-request cryptographic cost [Zhu et al \(2023\)](#); [Sikdar \(2026a\)](#). The reasoning is intuitive: validating a JSON Web Token means verifying a signature, signatures are cryptography, and cryptography is expensive relative to parsing. The conclusion usually drawn is that the cost should be *moved* — to a sidecar proxy, a gateway, or dedicated hardware — because specialised components do cryptographic work well.

We report that for a widely used JavaScript implementation the premise is wrong, and that our own earlier attempt to correct it was also wrong, in a way worth recording. That earlier work [Sikdar \(2026a\)](#) established that the signature primitive is a negligible share of the cost and attributed the remainder to converting key material from a string into a key object on every call. Measuring the conversion directly shows it costs $0.583\ \mu\text{s}$ — too small to be the explanation for anything.

The actual mechanism is a control-flow accident. Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed. Every validation therefore constructs and discards an OpenSSL parse failure, at $18.04\ \mu\text{s}$ against a total call cost of $29.92\ \mu\text{s}$.

Contributions

1. A decomposition, by two independent methods, locating the dominant cost of token validation in a discarded exception rather than in cryptography or in key conversion (Section 4).
2. Evidence that the magnitude of this cost is governed by the runtime’s bundled OpenSSL version rather than by containerisation, virtualisation or the JavaScript engine, using a runtime pair that separates OpenSSL from V8 (Section 5).
3. A survey of public code establishing that the pattern which avoids the cost is rare in practice — found at 7 call sites across 6 projects by a search designed to find it (Section 6).
4. A fix, together with a constraint on it: the discarded attempt is load-bearing for an existing key-type defence, and the natural form of the optimisation is therefore unsafe (Section 7).

2 Background and related work

What a validation call does.

A JSON Web Token carries a header, a payload and a signature over the first two [Jones et al \(2015\)](#). Validating one means decoding the first two segments, resolving key material into a form the cryptographic library accepts, and verifying the signature. The cost of the middle step — key resolution — is the subject of this paper, and it is the step most often left implicit when validation cost is discussed.

Where the cost is usually attributed.

Published measurement of layered enforcement in microservices concentrates on the transport and proxy layers. Zhu et al. [Zhu et al \(2023\)](#) decompose service-mesh sidecar overhead and show the dominant contributors depend on configuration and workload rather than being a fixed tax — a conclusion we reach independently, one layer up, for

application-level validation. Application-level token validation itself is thinly evidenced: it is commonly reported as a difference between scenario averages, which attributes to validation every difference between two runs. Our own earlier study did exactly that [Sikdar \(2026a\)](#), and Section 9 reports what that approach captured. We are not able to cite a study that measures the key-resolution step directly, which is part of why the misattribution has persisted; we state this as an absence we searched for rather than as a claim about the whole literature.

Why the measurement protocol matters here.

The methodological core of this paper is not new. Georges et al. [Georges et al \(2007\)](#) established that a single execution of a managed-runtime benchmark is not a measurement and that intervals must be computed across executions, not within one. Mytkowicz et al. [Mytkowicz et al \(2009\)](#) showed that environmental details unrelated to the code under study can produce effects larger than those being measured, and that such bias is easy to produce without any obvious mistake. Traini et al. [Traini et al \(2022\)](#) found steady state is frequently not reached at all and that developers misjudge when warmup ends. Our contribution is not to these methods but an instance of what they warn about: two of the errors we correct in Section 9 are our own, and both have the structure Mytkowicz et al. describe — a second variable moving unrecorded alongside the one under study.

Environment as a confounder.

Laaber et al. [Laaber et al \(2019\)](#) quantify how far cloud environments degrade

microbenchmark reliability, and Leitner and Cito [Leitner and Cito \(2016\)](#) characterise performance variation across public IaaS providers. Their concern is variance between nominally identical environments. Ours is adjacent and, we think, less examined: a *systematic* difference between environments that are not identical and are routinely treated as though they were — two container base images differing in a bundled library version.

Key-type confusion.

That a token declaring an HMAC algorithm must not be validated against asymmetric key material is long established; RFC 8725 [Sheffer et al \(2020\)](#) describes the attack and the mitigations, citing CVE-2015-9235. Section 7 does not report a new attack. It reports that an existing defence depends on the control flow whose cost this paper measures, so that removing the cost naively removes the defence.

3 Methodology

3.1 What is measured, and at what granularity

The unit of measurement is one operating-system process, not one call. Each process performs 40000 timed iterations of a single condition after a warmup of half that number, and contributes its median. Reported figures are medians of per-process medians, with percentile bootstrap intervals taken over processes.

This matters because the alternative is misleading in a specific way. An interval computed over the calls inside one process describes that process’s steady state and is far too narrow: it cannot observe the compilation decisions, inline-cache states and heap layouts that differ between runs of the same code,

which is precisely what makes a micro-benchmark hard to reproduce [Georges et al \(2007\)](#). Conditions in the decomposition of Section 4 were run in 15 separate processes of 40000 calls; conditions in the runtime matrix of Section 5 in 10 processes of 20000 calls. Dispersion between processes differs correspondingly: the largest between-process coefficient of variation is 3.5% on the host and 20.2% in the matrix, the latter on `probe_succeeds`. Matrix conditions are therefore noisier than host conditions by roughly a factor of 5.7, which is why the matrix is used for the large between-runtime contrasts and not for fine-grained comparisons within a runtime.

Conditions are interleaved within each round rather than run in blocks. Under block ordering, a thermal excursion or a background process lands entirely on whichever condition was executing and is indistinguishable from an effect; interleaved, it is distributed across conditions and appears as between-round variance, which the intervals then report.

The clock’s own cost is measured in the same process as the condition it accompanies and is reported rather than subtracted: 0.042 μ s against the smallest quantity we report, 0.583 μ s.

3.2 Decomposing the call

`jwt.verify()` is decomposed into the operations it performs, each timed in isolation under the same protocol: structural decode; the HMAC primitive with a string key and with a pre-parsed key; the symmetric key constructor; and the asymmetric key parse that the library attempts first, in both the case where it throws and the case where it succeeds.

3.3 Separating the runtime from the deployment

Our earlier work compared a host micro-benchmark against an in-container measurement and attributed the difference to deployment. Those two environments differed in more than deployment, so the comparison could not support the attribution. Here containerisation is held constant and the runtime is varied: four container images spanning two OpenSSL series and three V8 major versions, all on one machine, under one container engine.

The decisive pair is Node 18 and Node 20: they ship V8 10 and V8 11 respectively while sharing the OpenSSL 3.0 series. In stock images Node and OpenSSL versions otherwise co-vary, which would leave the attribution under-determined. Because that pairing carries the attribution, the runtime identity of each image — Node, OpenSSL and V8 version, libc, architecture and image digest — is itself recorded as a datum rather than asserted, and is checked against the versions each measurement row carried.

3.4 Independent confirmation

The decomposition above reaches its conclusion by subtracting conditions — the same inference style this paper criticises elsewhere. It is therefore checked against V8’s sampling profiler, which attributes self time to named frames with no arithmetic on our part.

3.5 Prevalence

Three measurements with different failure modes. Population counts from code search over public repositories; a sample of call sites classified by tracing the key argument within its file; and a targeted counter-search for the avoiding pattern,

Table 1 Cost of each operation in the validation path. Medians of per-process medians over 15 processes, 40000 calls each; 95% percentile bootstrap intervals.

Operation	Median (μ s)
verify(), string secret	29.92 [29.75, 30.12]
failed asymmetric parse	18.04 [17.85, 18.25]
successful asymmetric parse	13.83 [13.62, 13.92]
verify(), pre-parsed key	6.04 [5.96, 6.12]
HMAC, string key	2.21 [2.21, 2.25]
HMAC, pre-parsed key	2.17 [2.13, 2.19]
structural decode	1.29 [1.25, 1.33]
symmetric key constructor	0.583 [0.54, 0.58]
timer overhead	0.042 [0.04, 0.044]

fetches in full and adjudicated by hand. The third exists because a sample can only show a pattern is rare if it could have found it.

4 The mechanism

4.1 Where the time goes

Table 1 gives the decomposition on the host.

The two bold rows are the result. The conversion to which the cost has been attributed — including by us — costs 0.583 μ s. The failure that precedes it costs 18.04 μ s, which is 60% of the whole call.

Passing a pre-parsed key instead of a string reduces the call by 23.92 μ s [23.58, 24.00] over 15 paired rounds.

4.2 The parts account for most, but not all, of the penalty

If the penalty were exactly the discarded failure plus the conversion, the two would sum to it. They do not, quite:

observed penalty	23.92 μ s
failed parse + conversion	18.62 μ s
residual	5.29 μ s

The residual’s interval, [4.92, 5.50], excludes zero, so the two measured parts are not the whole story: they account for 77.9%, and roughly 5.29 μ s is the

library’s own type and claim checking along the non-pre-parsed path. We report this rather than absorb it into the head-

4.3 Confirmation without differencing

VS sampling profiler attributes self time to named frames directly. At 50000 iterations per run and 3 runs per environment — matched, because an earlier pass used different iteration counts per environment and the resulting percentages were not comparable — the asymmetric key parse accounts for a median 66.2% of self time on the host (65.3–67.4 across runs), 95.3% (95.1–95.6) under Node 18 and 61.5% (58.5–61.6) under Node 26. The frame does not appear among the top frames of the pre-parsed path in any environment. Two methods, one answer. We rely on the ordering and the dominance, not on the exact percentages: sampling attribution depends on rate and on what else the process is doing.

5 The magnitude is a property of the runtime

Table 2 varies the runtime with containerisation held constant.

Three things follow.

Containerisation is not isolated by this design, and we do not claim it.

A reader may notice that the uncontainerised host row is of the same order as the Node 26 container row (18.04 against 16.58 μ s). That comparison is confounded and should not be read as evidence about

Table 2 The same measurement across four container runtimes, on one machine, 10 processes each, 20000 calls per process. The host row is from Table 1.

Runtime	OpenSSL	V8	Failed parse (μ s)	Successful verify()
Node 18, musl	3.0.16	10	373.79	7.58
Node 20, musl	3.0.19	11	420.62	7.25
Node 26, musl	3.5.7	14	22.94	6.36
Node 26, glibc	3.5.7	14	16.58	6.14
host (no container)	—	—	18.04	6.04

6 How common is the pattern?

A defect that nothing triggers is a curiosity. Three measurements.

Population counts.

containerisation: the host runs macOS and the containers run Linux, so operating system, C library and Node build all change alongside the container boundary. Isolating containerisation would require a Linux-native baseline, which we did not obtain. The claims below concern only the four container rows, among which containerisation is held constant.

It is OpenSSL, not the JavaScript engine.

Node 18 and Node 20 ship V8 10 and 11 — different major versions of the engine — and share the OpenSSL 3.0 series. Both pay of order 373.79–420.62 μ s. Node 26 ships OpenSSL 3.5 and pays 22.94 μ s on the same libc. Because Node and OpenSSL versions covary in stock images, this attribution is normally underdetermined; the Node 20 runtime is what separates them.

The effect is confined to key parsing.

Across every environment measured, pre-parsed validation spans 1.26 $\times$, the HMAC primitive 1.47 $\times$ and structural decode 1.69 $\times$. The failed parse spans 25.36 $\times$. The successful parse moves with it (11.74 $\times$), so this is the cost of OpenSSL 3.0 key parsing in general rather than anything peculiar to the failure path.

Code search over public repositories, by language and import form (captured 2026-09-18). Both forms are counted in both languages: an earlier version of this analysis queried `require` in JavaScript and `import` in TypeScript only, which excluded ECMAScript modules in JavaScript — the largest single cell — altogether.

Lang.	Import form	Files	With API	%
JS	<code>require</code>	790528	29	0.0037
JS	<code>import</code>	524288	38	0.0072
TS	<code>require</code>	8288	0	0.0000
TS	<code>import</code>	333824	101	0.0303

Cells are reported separately rather than pooled: a file using both import forms would be counted twice by a sum, so the sum 1656928 is an upper bound on the union and the largest cell 790528 a lower bound. The rate varies between cells — from 0.0000% to 0.0303% — but no cell exceeds 0.0303%. Pooling the cells, which we do not rely on, would give 168 of 1656928, or 0.0101%: roughly one file in 9863.

These are search-engine estimates of file counts, not a census. They drift between captures, they cover public repositories only, and they include

forks, vendored dependencies and teaching material. They are reported to establish an order of magnitude and should not be read as a population proportion.

Sampled call sites.

327 call sites across 247 distinct repositories, one file per repository so that no single project dominates. Environment variables account for 176, untraced identifiers and member expressions 93, string literals 52, file contents 4 and buffers 2. Pre-parsed keys account for 0.

The first version of this classifier recognised only the conventional `jwt.verify()` spelling, which filtered the corpus toward one import idiom. It was re-run over the previously unclassified residue with a matcher that resolves the local binding of the import — covering arbitrary aliases, namespace imports, destructured and renamed imports, and direct calls on the `require` expression. That recovered 12 further call sites, included above, and 0 of them pass a pre-parsed key. No file in this corpus contains the pre-parsing API at all, in any spelling, so the zero reflects the pattern’s absence from this sample rather than a classifier that cannot see it.

The corpus is third-party source and is not redistributed. To make that claim checkable rather than merely asserted, the replication package ships a manifest of all 502 files examined across 457 repositories, recording for each the repository, path, git blob hash, SHA-256 of the exact bytes analysed, and — where applicable — the adjudication verdict. A reader can re-fetch any file, hash it, and establish either that they hold what we held or that it has since changed [Sikdar \(2026b\)](#).

Incidentally, 309 of 327 call sites (94%) pass no algorithm restriction. This

is not what the survey set out to measure, but it bears directly on Section 7.

Counter-search, adjudicated by hand.

Every file co-occurring the library with the pre-parsing API was fetched: 191 files, of which 120 contain a validation call. An automated pass flagged 13 as passing a pre-parsed key; all were then read. Of these, 5 are false positives — projects defining their own helper function of the same name, one of which returns a string — and 1 could not be resolved within the file. Genuine: 7 call sites across 6 projects.

7 A fix, and a constraint on it

The fix is to dispatch on the token’s declared algorithm before attempting the asymmetric parse, so that an HMAC token with an ordinary string secret is resolved directly. Applied, this takes the call from 29.92 μ s to 7.21 μ s, a factor of 4.15 and within 1.21 μ s of supplying a pre-parsed key, with no change in calling code.

The constraint is the more important half. The discarded attempt is not purely wasteful: the *success* of that parse on asymmetric material is what produces a key object whose type a later check depends on, and that check is what prevents symmetric and asymmetric key material from being interchanged. A change that dispatches on the declared algorithm while relaxing which key material may take the fast path alters which inputs reach that check.

The fix must therefore be narrow: the fast path is restricted to string secrets that cannot be asymmetric key material, so that anything which might be resolves exactly as it does today.

We tested this against the library’s own suite rather than only against assertions of our own. Cloned at the release tag, the patch applies cleanly and the suite is unchanged: 511 passing and 0 failing unpatched, 511 and 0 patched. Removing the material restriction — the form of the optimisation we argue against — produces 2 failures, both in the suite’s `wrong_alg` tests, which exist for exactly this property.

That last result is worth stating plainly because it reduces our own contribution. The upstream suite already constrains this: a contributor proposing the unrestricted optimisation would be caught by tests they are required to run. We therefore do not claim a new regression test. What we claim is narrower — that the cost documented in Section 4 cannot be removed by the obvious route, and that the reason is a property the maintainers already test for but which is not evident from the code being changed.

The independent defence against this class of confusion is for the caller to restrict permitted algorithms [Sheffer et al \(2020\)](#) — and 94% of surveyed call sites do not.

8 Implications for practice

Pre-parse key material once, at startup.

On the runtimes measured here this removes 23.92 μ s per validation [23.58, 24.00] and requires no architectural change, no new component and no movement of a trust boundary. It is a one-line change at the point where the secret is read.

Do not reach for an architectural remedy first.

A cost believed to be cryptographic invites relocation — to a sidecar, a gateway, dedicated hardware. Section 7 is the argument against treating that as the default response: the cost here is not cryptographic, it is largely avoidable in place, and the remedy that moves validation elsewhere also moves a trust boundary. An optimisation that alters which key material reaches a type check is not a performance change with a security footnote; it is a security change.

Pin the algorithm.

Independently of anything in this paper, callers should restrict permitted algorithms [Sheffer et al \(2020\)](#). 94% of the call sites we surveyed do not, which leaves them relying entirely on library-internal key typing for a property they could assert directly in one argument.

Treat the base image as part of the measurement.

The same code, the same library and the same architecture differ by 25.36 $\times$ on this path across the container images we measured, because they pin different OpenSSL versions. A performance figure quoted without the base image it was obtained on is not reproducible, and a regression that appears after a base-image bump may have nothing to do with the application.

9 Correcting our own earlier report

Our earlier manuscript reported that the dominant term was key material converted from a string on every call. That is wrong: the conversion costs 0.583 μ s. It also explained an asymmetry between

HMAC and RSA costs as conversion being dearer than parsing a PEM, which is backwards — a successful asymmetric parse costs 13.83 μ s against 0.583 μ s for the conversion. The real asymmetry is that the RSA path’s parse *succeeds*, making its cost useful work, while the HMAC path’s fails, making its cost waste.

That manuscript further compared a host microbenchmark against an in-container measurement and attributed the difference to deployment. Section 5 shows the comparison spanned two OpenSSL major series, so it could not support that attribution. It also stated a runtime version for the system under test that was read from the host rather than from the container: it reported Node 26.6.0 where the service ran Node 18.20.8.

We record these rather than revise them silently because the errors share a structure worth naming: in each case a quantity was attributed to the variable we were interested in, when a second variable had changed at the same time and had not been recorded at the granularity that would have revealed it.

10 Threats to validity

One machine, one instruction set.

Every measurement in this paper comes from a single ARM64 host: an Apple silicon laptop, its container runtimes under one engine. We have **not** verified any of this on x86_64.

We state what we expect and why, and then state that we did not test it. The mechanism is a property of a library’s API contract — an asymmetric parse attempted before a symmetric one, with the failure discarded — and of the cost of key parsing in a particular OpenSSL series. Neither is a property of an instruction set, and we know of no reason for the

control flow to differ on x86_64. But an expectation is not a measurement. The magnitude in particular depends on how OpenSSL’s key parsing performs on a given platform, and that is exactly the kind of quantity that does vary with architecture. Until the following is filled in, no claim in this paper should be read as holding beyond ARM64: **[MISSING: xEightSixReplication]**.

One library, one version.

The mechanism is specific to one JavaScript implementation at one version. Whether other implementations or other language ecosystems attempt key resolution in the same order is unmeasured.

The prevalence survey measures files, not systems.

A project that constructs a pre-parsed key in one file and validates in another is invisible to file-level analysis, and this is the strongest threat to Section 6. Within-file tracing found no such case, which is weak evidence against it. Code search covers public repositories only and includes forks, vendored dependencies and teaching material; the numerator is inflated in the same way as the denominator, but the ratio is not a clean population proportion and should not be read as one. Public code also skews toward small projects, and the 6 projects found using the API correctly are all small.

The adjudication had a single rater with an interest in the outcome.

The 13 automatically flagged call sites were read and classified by the author, who is not disinterested: the paper’s claim is stronger the more of them are false positives, and 5 were classified that

way. There was no second rater, no blinding and therefore no inter-rater agreement to report. The per-case verdicts, the evidence for each and the hashes of the files they were read from are published so the adjudication can be repeated by someone else, which is a weaker guarantee than having done so.

Query selection shapes the sample, and did so once.

An earlier iteration of the call-site sample included a query term naming environment variables, which biased the corpus toward the very category under investigation. That query was removed and the sample regenerated. We report it because the error is invisible in the result: a biased query returns a plausible corpus, and nothing downstream distinguishes it from an unbiased one.

A search tool failed silently during this study.

The GitHub CLI's code-search subcommand returns an empty result for multi-term queries that the underlying REST API answers normally. The counter-search of Section 6 initially returned nothing through that path — a result indistinguishable from the pattern being absent, which is what the counter-search exists to test. It was caught by checking that the counter-search queries appeared in the corpus at all. Studies using code search should verify that each query contributed to the corpus rather than trusting that it ran.

Dispersion is not uniform across conditions.

The claim in Section 5 that non-parsing operations are stable across runtimes rests partly on conditions whose between-process variation is among the highest we

measured: the noisiest matrix condition at 20.2% is an HMAC measurement. The spans reported there are ratios of medians and the between-runtime differences they describe are one to two orders of magnitude larger than this dispersion, but the stability claim is correspondingly weaker than the difference claim.

Absolute figures are from a shared laptop.

The host also ran a desktop environment. Contention inflates a duration rather than deflating it, so the reported costs are upper bounds and the comparisons between conditions — which is where every claim here rests — are unaffected in direction. Absolute values should not be compared against dedicated hardware.

The host's cryptographic library moved mid-study, and we could not prove from the record that it had not.

Installing an unrelated command-line tool upgraded the host's OpenSSL beneath an unchanged Node binary, which links it dynamically; `node --version` was unchanged throughout. The host decomposition of Section 4 precedes the upgrade by wall-clock timestamp and the container measurements bundle their own OpenSSL, so we believe no reported figure is affected — but that host run recorded only the Node version, not the OpenSSL version, so the inference rests on run timestamps rather than on the measurement record itself. We report this as a limitation of the host rows and not as a resolved issue. The per-measurement OpenSSL field was added in response, which is why the runtime matrix carries it and the host decomposition does not. The recommendation we

draw — record the identity of the component whose behaviour is being measured at the granularity at which it can change — is one this study failed to follow before it needed it, not one it demonstrates.

11 Conclusion

The per-request cost of token validation in this library is real, is worth attending to, and is not cryptographic. It is not key conversion either. It is a parse attempted in the wrong order, whose failure is constructed and thrown away on every request, and whose price is set by the OpenSSL version a base image happens to pin — varying $25.36\times$ across runtimes while the rest of the validation path varies by at most $1.26\times$. The pattern that avoids it is rare in public code: a search targeted at finding it located 7 genuine call sites across 6 projects.

The methodological lesson is not that one should measure in deployment. It is narrower and, we think, more useful: attribute a cost only to a variable you recorded at the granularity at which it can change. Both errors we correct here — our own, and the field’s — have that shape.

Declarations

Data and code availability. All measurements, the instrumentation, the analysis and the scripts that produce every numeric value in this paper are in the replication package. Every number is generated from a file; none is entered by hand. Quantities that were not measured appear as red placeholders rather than as absences.

Competing interests. The author declares no competing interests.

References

- Georges A, Buytaert D, Eeckhout L (2007) Statistically rigorous Java performance evaluation. In: Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA ’07), <https://doi.org/10.1145/1297027.1297033>
- Jones MB, Bradley J, Sakimura N (2015) JSON web token (JWT). Request for Comments RFC 7519, Internet Engineering Task Force, <https://doi.org/10.17487/RFC7519>
- Laaber C, Scheuner J, Leitner P (2019) Software microbenchmarking in the cloud. how bad is it really? Empirical Software Engineering <https://doi.org/10.1007/s10664-019-09681-1>
- Leitner P, Cito J (2016) Patterns in the chaos—a study of performance variation and predictability in public IaaS clouds. ACM Transactions on Internet Technology <https://doi.org/10.1145/2885497>
- Mytkowicz T, Diwan A, Hauswirth M, et al (2009) Producing wrong data without doing anything obviously wrong! In: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’09), <https://doi.org/10.1145/1508244.1508275>
- Sheffer Y, Hardt D, Jones MB (2020) JSON web token best current practices. Request for Comments RFC 8725, Internet Engineering Task Force, <https://doi.org/10.17487/RFC8725>

Sikdar K (2026a) Quantifying the performance cost of API security in cloud-native microservices: An empirical study of TLS, mTLS, and JWT validation. Unpublished manuscript. The per-request attribution reported there is corrected by the present paper; see Section 9.

Sikdar K (2026b) Replication package: key-resolution cost in token validation. <https://github.com/Katha-Sikdar/microservices-api>, measurements under `psao-artifact/data/runs/`; survey corpus manifest at `psao-artifact/survey/data/corpus_manifest.csv`

Traini L, Cortellessa V, Di Pompeo D, et al (2022) Towards effective assessment of steady state performance in Java software: Are we there yet? Empirical Software Engineering <https://doi.org/10.1007/s10664-022-10247-x>

Zhu X, She G, Xue B, et al (2023) Dissecting overheads of service mesh sidecars. In: Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23), <https://doi.org/10.1145/3620678.3624652>